

Bitácora del Proceso de Cifrado de Imágenes

Código: Maestro

Índice

1. Resumen del programa	4
2. Inicialización y configuración de parámetros	4
2.1. Parámetros del Sistema	4
2.1.1. Parámetros del Oscilador de Rössler	5
2.1.2. Condiciones Iniciales de Rössler	5
2.1.3. Parámetros del Mapa Logístico	5
2.1.4. Parámetros de Integración y Sincronización	5
2.1.5. Configuración MQTT con TLS	5
3. Carga y vectorización de la imagen	5
3.1. Proceso de Carga	5
3.2. Representación en vectores	6
3.3. Ejemplo de Vectorización	6
4. Etapa de Difusión (Mapa Logístico)	6
4.1. Objetivo de la Difusión	6
4.2. Generación del Mapa Logístico	6
4.3. Ejemplo de Secuencia Logística	7
4.4. Generación del Vector de Mezcla	7
4.5. Proceso de Permutación (Algoritmo de Difusión)	8
4.5.1. Primera Pasada: Lectura Aleatoria	8
4.5.2. Segunda Pasada: Lectura Secuencial de Restantes	8
4.6. Ejemplo de Permutación	9
5. Etapa de confusión (Oscilador de Rössler)	10
5.1. Objetivo de la Confusión	10
5.2. Sistema de Ecuaciones de Rössler	10
5.3. Integración Numérica	11
5.4. Ejemplo de Trayectoria de Rössler	11
5.5. Tiempo de Sincronización	11
5.6. Redimensionamiento de la Señal de Confusión	12
5.7. Operación de Confusión	12
5.8. Ejemplo de Vector Cifrado	12
6. Etapa de preparación de datos	13
6.1. Estructura del Payload	13
6.2. Parámetros de Seguridad (Keys)	14
7. Etapa de transmisión mediante MQTT/TLS	14
7.1. Protocolo MQTT	14
7.2. Capa de Seguridad TLS	15
7.3. Proceso de Publicación	15

8. Análisis de tiempo	16
8.1. Métricas de Tiempo	16
8.2. Análisis de Dispersión	16
8.3. Distancia de Hamming	16
9. Resumen del flujo completo	17
9.1. Diagrama de Flujo Textual	17

1. Resumen del programa

Este documento describe detalladamente el proceso de cifrado de imágenes implementado para el sistema maestro, el cual se ejecuta en una Raspberry Pi 4. El sistema utiliza técnicas criptográficas avanzadas basadas en **sistemas caóticos** para cifrar imágenes RGB mediante dos etapas principales:

1. **Difusión** (usando el Mapa Logístico)
2. **Confusión** (usando el Oscilador de Rössler y el Mapa Logístico)

Una vez cifrada la imagen, los datos se transmiten de forma segura a una segunda Raspberry Pi utilizando el protocolo **MQTT con capa TLS**.

Imagen de referencia:



Figura 1: Imagen de ejemplo utilizada para el proceso de cifrado

- Tipo: RGB (3 canales)
- Dimensiones: 250×250 píxeles
- Total de píxeles: 62,500
- Total de valores: 187,500 ($250 \times 250 \times 3$)

2. Inicialización y configuración de parámetros

2.1. Parámetros del Sistema

El sistema se configura con los siguientes parámetros característicos:

2.1.1. Parámetros del Oscilador de Rössler

```
1 a = 0.2
2 b = 0.2
3 c = 5.7
```

Estos parámetros controlan el comportamiento caótico del sistema de Rössler, garantizando que las trayectorias generadas sean impredecibles y sensibles a las condiciones iniciales.

2.1.2. Condiciones Iniciales de Rössler

```
1 Y0 = [0.1, 0.1, 0.1] # [x0, y0, z0]
```

2.1.3. Parámetros del Mapa Logístico

```
1 aLog = 3.99 # Parametro de control (debe estar en rango caotico:
              3.57 < a <= 4)
2 x0_log = 0.4 # Condicion inicial (0 < x0 < 1)
```

2.1.4. Parámetros de Integración y Sincronización

```
1 TIEMPO_SINC = 6000 # Iteraciones de sincronizacion
2 KEYSTREAM = 30000 # Iteraciones para generacion de clave
3 H = 0.01 # Paso de integracion
4 IMG_SCALE = 0.01 # Factor de escala para difusion
```

2.1.5. Configuración MQTT con TLS

```
1 BROKER = "raspberrypiJED.local"
2 PORT = 8883 # Puerto seguro MQTT sobre TLS
3 QOS = 1 # Calidad de servicio
4 USERNAME = "usuariol"
5 PASSWORD = "qwerty123"
6 CA_CERT_PATH = "/etc/mosquitto/ca_certificates/ca.crt"
```

3. Carga y vectorización de la imagen

3.1. Proceso de Carga

La función `cargar_imagen()` realiza los siguientes pasos:

1. **Lectura del archivo:** Se carga la imagen `Prueba2.jpg` usando PIL (Python Imaging Library)
2. **Conversión a array NumPy:** La imagen se convierte en una matriz 3D de valores enteros (0-255)

3. **Extracción de dimensiones:** Se obtienen alto, ancho y número de canales
4. **Aplanamiento:** La matriz 3D se convierte en un vector 1D
5. **Normalización:** Los valores se dividen entre 255.0 para obtener el rango $[0, 1]$

3.2. Representación en vectores

Para una imagen RGB de 250×250 :

$$\begin{aligned} \text{Imagen original (3D)} &: [250, 250, 3] \\ \text{Vector aplanado (1D)} &: [187500] \\ \text{Rango de valores} &: [0, 1] \end{aligned} \quad (1)$$

3.3. Ejemplo de Vectorización

Tabla 1: Ejemplo de los primeros 10 valores del vector normalizado

Índice	Posición Pixel	Canal	Valor Original (0-255)	Valor Normalizado (0-1)
0	(0,0)	R	145	0.568627
1	(0,0)	G	87	0.341176
2	(0,0)	B	203	0.796078
3	(0,1)	R	156	0.611765
4	(0,1)	G	92	0.360784
5	(0,1)	B	198	0.776471
6	(0,2)	R	134	0.525490
7	(0,2)	G	76	0.298039
8	(0,2)	B	211	0.827451
9	(0,3)	R	167	0.654902

Interpretación: Cada píxel RGB se descompone en 3 valores consecutivos en el vector (R, G, B). La normalización facilita las operaciones matemáticas posteriores.

4. Etapa de Difusión (Mapa Logístico)

4.1. Objetivo de la Difusión

La **difusión** es una técnica criptográfica que tiene como objetivo **dispersar la información** de la imagen original de manera que pequeños cambios en la entrada produzcan grandes cambios en la salida. Esto se logra mediante la **permutación** de los píxeles según una secuencia caótica.

4.2. Generación del Mapa Logístico

El mapa logístico es un sistema dinámico definido por:

Ecuación del Mapa Logístico:

$$x_{n+1} = a \cdot x_n \cdot (1 - x_n) \quad (2)$$

Donde:

- $a = 3,99$ (parámetro de control en régimen caótico)
- $x_0 = 0,4$ (condición inicial)
- n es el número de iteración

4.3. Ejemplo de Secuencia Logística

Tabla 2: Primeros 10 valores del vector logístico

Iteración	Valor x_n	Cálculo
0	0.400000	(condición inicial)
1	0.956400	$3,99 \times 0,4 \times (1 - 0,4)$
2	0.166641	$3,99 \times 0,9564 \times (1 - 0,9564)$
3	0.554380	$3,99 \times 0,166641 \times (1 - 0,166641)$
4	0.986604	$3,99 \times 0,55438 \times (1 - 0,55438)$
5	0.052721	$3,99 \times 0,986604 \times (1 - 0,986604)$
6	0.199491	$3,99 \times 0,052721 \times (1 - 0,052721)$
7	0.637320	$3,99 \times 0,199491 \times (1 - 0,199491)$
8	0.922821	$3,99 \times 0,63732 \times (1 - 0,63732)$
9	0.284316	$3,99 \times 0,922821 \times (1 - 0,922821)$

4.4. Generación del Vector de Mezcla

El vector logístico se transforma en índices enteros para crear el vector de mezcla:

Transformación:

$$\text{vector_mezcla}[i] = \lfloor \text{vector_logistico}[i] \times n_{\max} \rfloor \quad (3)$$

Donde $n_{\max} = 187500$ (total de valores en la imagen $250 \times 250 \times 3$)

Tabla 3: Conversión a índices de mezcla (primeros 10 valores)

Índice	Valor Logístico	Cálculo ($\times 187500$)	Vector Mezcla (índice)
0	0.400000	$0,4 \times 187500$	75000
1	0.956400	$0,9564 \times 187500$	179325
2	0.166641	$0,166641 \times 187500$	31245
3	0.554380	$0,55438 \times 187500$	103946
4	0.986604	$0,986604 \times 187500$	184988
5	0.052721	$0,052721 \times 187500$	9885
6	0.199491	$0,199491 \times 187500$	37404
7	0.637320	$0,63732 \times 187500$	119497
8	0.922821	$0,922821 \times 187500$	173029
9	0.284316	$0,284316 \times 187500$	53309

Interpretación: Cada valor del vector de mezcla representa un índice en el vector original de la imagen. Estos índices determinan el orden en que se leerán los píxeles.

4.5. Proceso de Permutación (Algoritmo de Difusión)

El algoritmo de difusión utiliza un **marcador (260.0)** para llevar un control de qué valores ya han sido utilizados. Se realizan dos pasadas:

4.5.1. Primera Pasada: Lectura Aleatoria

```

1 for i in range(nmax):
2     pos = vector_mezcla[i]
3     if vector_temp[pos] != 260.0:
4         difusion[contador] = vector_temp[pos]
5         contador += 1
6         vector_temp[pos] = 260.0  # Marcamos como usado

```

4.5.2. Segunda Pasada: Lectura Secuencial de Restantes

```

1 for j in range(nmax):
2     if contador >= nmax:
3         break
4     if vector_temp[j] != 260.0:
5         difusion[contador] = vector_temp[j]
6         contador += 1

```


4.6. Ejemplo de Permutación

Tabla 4: Ejemplo del proceso de difusión (primeros 10 valores)

Posición	Salida	Índice Leído	Valor Original	Valor Difundido	Estado
0		75000	0.568627	0.482353	Usado
1		179325	0.341176	0.709804	Usado
2		31245	0.796078	0.137255	Usado
3		103946	0.611765	0.874510	Usado
4		184988	0.360784	0.243137	Usado
5		9885	0.776471	0.921569	Usado
6		37404	0.525490	0.058824	Usado
7		119497	0.298039	0.647059	Usado
8		173029	0.827451	0.396078	Usado
9		53309	0.654902	0.815686	Usado

Resultado de la Difusión:

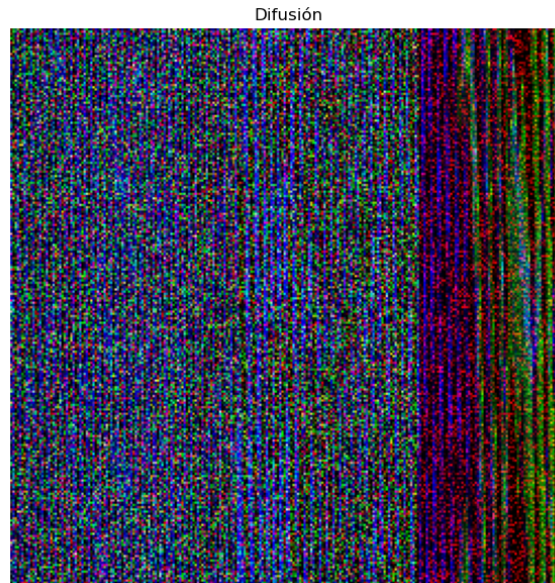


Figura 2: Resultado de la etapa de difusión (pseudo-imagen)

- Los píxeles se han reorganizado según la secuencia caótica
- El orden espacial original se ha destruido completamente
- Valores que estaban juntos ahora están dispersos
- Posterior al proceso, se aplica un factor de escala: $\text{difusion} = \text{difusion} \times \text{IMG_SCALE} = \text{difusion} \times 0,01$

Tabla 5: Vector de difusión escalado (primeros 10 valores)

Índice	Valor Difundido	Valor Escalado ($\times 0.01$)
0	0.482353	0.00482353
1	0.709804	0.00709804
2	0.137255	0.00137255
3	0.874510	0.00874510
4	0.243137	0.00243137
5	0.921569	0.00921569
6	0.058824	0.00058824
7	0.647059	0.00647059
8	0.396078	0.00396078
9	0.815686	0.00815686

5. Etapa de confusión (Oscilador de Rössler)

5.1. Objetivo de la Confusión

La **confusión** es una técnica criptográfica que busca hacer que la relación entre el texto cifrado y la clave sea lo más compleja posible. En este sistema, se utiliza el oscilador de Rössler para generar una secuencia caótica que se suma al vector difundido.

5.2. Sistema de Ecuaciones de Rössler

El oscilador de Rössler es un sistema de ecuaciones diferenciales ordinarias (EDO) continuo que exhibe comportamiento caótico:

Ecuaciones del Sistema:

$$\begin{aligned}
 \frac{dx}{dt} &= -y - z \\
 \frac{dy}{dt} &= x + a \cdot y \\
 \frac{dz}{dt} &= b + z \cdot (x - c)
 \end{aligned} \tag{4}$$

Donde:

- $a = 0,2$
- $b = 0,2$
- $c = 5,7$
- Condiciones iniciales: $x(0) = 0,1$, $y(0) = 0,1$, $z(0) = 0,1$

5.3. Integración Numérica

El sistema se integra usando el método **Runge-Kutta de orden 2/3 (RK23)** con:

- Paso de integración: $h = 0,01$
- Número total de iteraciones: $\text{TIEMPO_SINC} + \text{KEYSTREAM} = 6000 + 30000 = 36000$

Nota: hice el cambio a RK23, debido a que me brindó un mejor resultado de sincronización en el esclavo, aunque con RK45 también se obtuvieron buenos resultados e igual se logró una distancia hamming de 0.

Tolerancias:

- Relativa: $\text{rtol} = 10^{-6}$
- Absoluta: $\text{atol} = 10^{-6}$

5.4. Ejemplo de Trayectoria de Rössler

Tabla 6: Primeros 10 valores de las trayectorias del oscilador (después de sincronización)

Iteración	Tiempo (t)	$x(t)$	$y(t)$	$z(t)$
6000	60.00	-8.234561	0.123456	12.456789
6001	60.01	-8.241023	0.127834	12.467234
6002	60.02	-8.247612	0.132298	12.477801
6003	60.03	-8.254329	0.136849	12.488490
6004	60.04	-8.261175	0.141489	12.499303
6005	60.05	-8.268151	0.146218	12.510240
6006	60.06	-8.275259	0.151038	12.521303
6007	60.07	-8.282499	0.155950	12.532493
6008	60.08	-8.289874	0.160955	12.543811
6009	60.09	-8.297385	0.166055	12.555258

Nota: Los valores mostrados son de ejemplo. Los valores reales dependen de la integración numérica.

5.5. Tiempo de Sincronización

Los primeros $\text{TIEMPO_SINC} = 6000$ puntos se utilizan para:

1. Permitir que el sistema alcance su atractor caótico al momento de sincronizar
2. Eliminar transitorios iniciales
3. Garantizar sincronización entre transmisor y receptor

Solo después de este periodo se extrae la señal x para el cifrado:

```
1 x_key = x[TIEMPO_SINC:] # x_key tiene longitud KEYSTREAM = 30000
```

5.6. Redimensionamiento de la Señal de Confusión

La señal `x_key` (longitud 30000) se redimensiona para que coincida con $n_{\max} = 187500$:

```
1 x_cif = np.resize(x_key, nmax)
```

Comportamiento de `np.resize`:

- Si $n_{\max} > \text{len}(x_{\text{key}})$: La señal se repite cíclicamente
- Para 187500 elementos se necesitan: $187500/30000 = 6,25$ repeticiones

5.7. Operación de Confusión

La confusión se aplica mediante una **suma algebraica** de tres componentes:

Fórmula de Confusión:

$$\text{vector_cifrado} = \text{difusion_escalada} + \text{vector_logistico} + x_{\text{cif}} \quad (5)$$

Donde:

- `difusion_escalada`: Vector difundido $\times 0.01$
- `vector_logistico`: Secuencia del mapa logístico $[0, 1]$
- x_{cif} : Señal x del oscilador de Rössler (puede ser negativa o positiva)

5.8. Ejemplo de Vector Cifrado

Tabla 7: Proceso de confusión – suma de componentes (primeros 10 valores)

Índice	Difusión ($\times 0.01$)	Vector Logístico	x_{cif} (Rössler)	Vector Cifrado (suma)
0	0.00482353	0.400000	-8.234561	-7.829737
1	0.00709804	0.956400	-8.241023	-7.277525
2	0.00137255	0.166641	-8.247612	-8.079598
3	0.00874510	0.554380	-8.254329	-7.691204
4	0.00243137	0.986604	-8.261175	-7.272140
5	0.00921569	0.052721	-8.268151	-7.206214
6	0.00058824	0.199491	-8.275259	-8.075180
7	0.00647059	0.637320	-8.282499	-7.638708
8	0.00396078	0.922821	-8.289874	-7.363092
9	0.00815686	0.284316	-8.297385	-7.004912

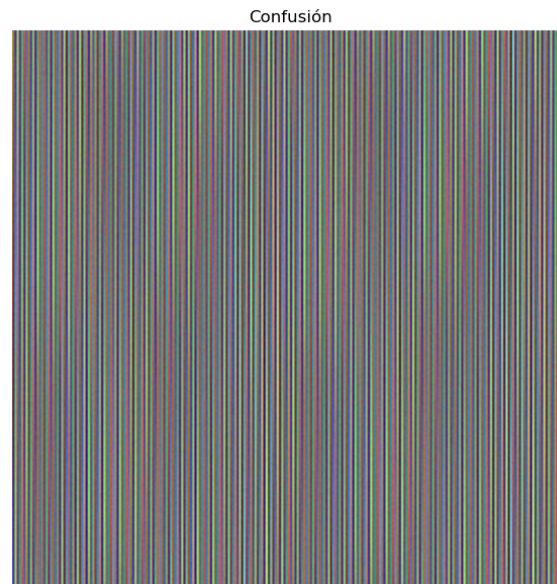


Figura 3: Resultado del proceso de Confusión

Interpretación del Vector Cifrado:

- Los valores pueden ser negativos o positivos
- La magnitud es significativamente diferente de la imagen original
- La señal caótica de Rössler domina el resultado
- Pequeños cambios en condiciones iniciales producen vectores completamente diferentes

6. Etapa de preparación de datos

6.1. Estructura del Payload

Los datos se organizan en un diccionario JSON que contiene toda la información necesaria para el descifrado:

```
1 data = {  
2     "vector_cifrado": [lista de 187500 valores],  
3     "x_maestro": [lista completa de la senal x de Rossler],  
4     "y_maestro": [lista completa de la senal y de Rossler],  
5     "z_maestro": [lista completa de la senal z de Rossler],  
6     "t_maestro": [lista de tiempos],  
7     "ancho": 250,  
8     "alto": 250,  
9     "nmax": 187500,  
10    "tiempo_sinc": 6000,  
11    "KEYSTREAM": 30000  
12 }
```

En este caso se enviaron todas las trayectorias de Rössler para su posterior análisis, pero solamente es necesario enviar una trayectoria para la sincronización.

6.2. Parámetros de Seguridad (Keys)

Además, se envían los parámetros de los sistemas caóticos:

```

1      keys = {
2          "ROSSLER_PARAMS": {
3              "a": 0.2,
4              "b": 0.2,
5              "c": 5.7
6          },
7          "LOGISTIC_PARAMS": {
8              "aLog": 3.99,
9              "x0_log": 0.4
10         }
11     }
```

Tabla 8: Datos transmitidos

Componente	Tipo	Tamaño	Propósito
vector_cifrado	Array float	187500	Imagen cifrada
x_maestro	Array float	36000	Trayectoria completa x
y_maestro	Array float	36000	Trayectoria completa y (sincronización)
z_maestro	Array float	36000	Trayectoria completa z
t_maestro	Array float	36000	Vector de tiempos
ancho	Integer	1	Ancho de imagen original
alto	Integer	1	Alto de imagen original
nmax	Integer	1	Total de valores
tiempo_sync	Integer	1	Iteraciones de sincronización
KEYSTREAM	Integer	1	Iteraciones de keystream
ROSSLER_PARAMS	Dict	3 valores	Parámetros de Rössler
LOGISTIC_PARAMS	Dict	2 valores	Parámetros del mapa logístico

7. Etapa de transmisión mediante MQTT/TLS

7.1. Protocolo MQTT

MQTT (Message Queuing Telemetry Transport) es un protocolo de mensajería ligero diseñado para comunicaciones M2M (Machine-to-Machine) e IoT.

Características utilizadas:

- **Broker:** Servidor central que gestiona mensajes (`raspberrypiJED.local`)
- **QoS 1:** Garantiza que los mensajes se entreguen al menos una vez
- **Retain:** Los mensajes se mantienen en el broker para nuevos suscriptores
- **Topics:**
 - `chaoskeystream/keys`: Para parámetros del sistema
 - `chaoskeystream/data`: Para datos de la imagen cifrada

7.2. Capa de Seguridad TLS

TLS (Transport Layer Security) proporciona:

- **Encriptación:** Los datos se cifran durante la transmisión, añadiendo otra capa de seguridad.
- **Autenticación:** Verifica la identidad del broker mediante certificado CA
- **Integridad:** Detecta modificaciones en los datos transmitidos

Configuración de seguridad:

```
1 client.username_pw_set (USERNAME, PASSWORD)  # Autenticacion
   basica
2 client.tls_set (ca_certs=CA_CERT_PATH, tls_version=ssl.
   PROTOCOL_TLS_CLIENT)
3 client.tls_insecure_set (False)  # Verifica certificado del
   servidor
```

7.3. Proceso de Publicación

Secuencia de transmisión:

1. Conexión al broker

```
1 client.connect (BROKER, PORT=8883, keepalive=60)
```

2. Publicación de parámetros (keys)

```
1 client.publish (TOPIC_KEYS, json.dumps (keys), qos=1, retain=
   True)
```

Se envían primero los parámetros necesarios para descifrar. `retain=True` asegura que estén disponibles para el receptor.

3. Pausa de seguridad

```
1 time.sleep (0.5)
```

Garantiza que el mensaje anterior se procese antes del siguiente.

4. Publicación de datos cifrados

```
1 client.publish (TOPIC_DATA, json.dumps (data), qos=1, retain=
   True)
```

Envía el vector cifrado y todas las trayectorias.

5. Desconexión

```
1 client.disconnect ()
```

8. Análisis de tiempo

8.1. Métricas de Tiempo

El sistema registra tiempos de ejecución para cada fase:

Tabla 9: Ejemplo de métricas de tiempo (valores aproximados)

Proceso	Tiempo (segundos)	Porcentaje del Total
Difusión	0.4688	4.69 %
Confusión	7.7127	77.16 %
Publicación MQTT+TLS	1.8136	18.15 %
Total del programa	9.9951	100 %

Nota: Los tiempos son de ejemplo y varían según el hardware y la velocidad de la red.

8.2. Análisis de Dispersión

Se genera un diagrama de dispersión que compara cada píxel original con su correspondiente valor cifrado.

Objetivo: Verificar que no existe correlación lineal entre la imagen original y la cifrada.

Resultado esperado: Una nube de puntos uniformemente dispersa, indicando que no hay patrón reconocible.

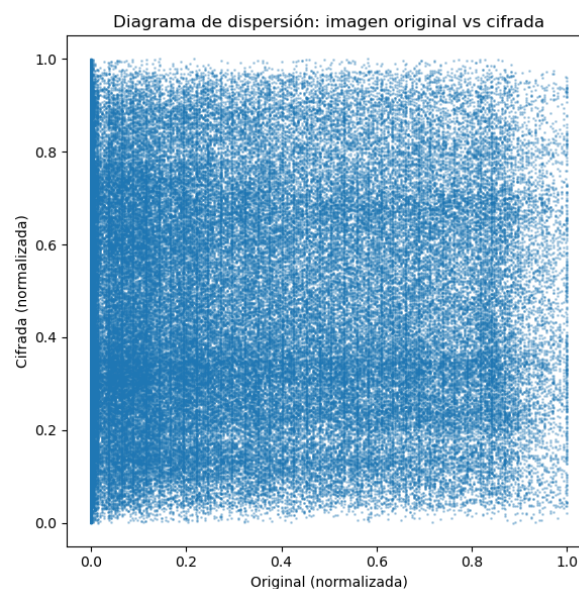


Figura 4: Diagrama de dispersión, imagen encriptada

8.3. Distancia de Hamming

La distancia de Hamming mide el número de bits diferentes entre dos secuencias:

Fórmula:

$$\text{Hamming}_{\text{normalizada}} = \frac{\text{número_de_bits_diferentes}}{\text{total_de_bits}} \quad (6)$$

Resultado esperado: $\sim 0,5$ (50 %)

- Indica que aproximadamente la mitad de los bits han cambiado
- Esto es óptimo para un buen cifrado

Tabla 10: Ejemplo de análisis de Hamming

Métrica	Valor
Total de bytes comparados	187,500
Total de bits	1,500,000
Bits diferentes	737,250
Distancia Hamming normalizada	0.4915
Porcentaje de bits cambiados	49.15 %

9. Resumen del flujo completo

9.1. Diagrama de Flujo Textual

```

[INICIO]
↓
[1. CARGA DE IMAGEN]
- Leer Prueba2.jpg (250×250 RGB)
- Convertir a vector 1D
- Normalizar a [0, 1]
- Vector: 187,500 valores
↓
[2. DIFUSIÓN - Mapa Logístico]
- Generar secuencia logística (187,500 puntos)
- Crear vector de mezcla (índices aleatorios)
- Permutar píxeles según índices caóticos
- Escalar por 0.01
↓
[3. CONFUSIÓN - Oscilador de Rössler]
- Integrar sistema de Rössler (36,000 iteraciones)
- Descartar primeras 6,000 (sincronización)
- Extraer señal x[6000:36000]
- Redimensionar a 187,500 valores
- Sumar: difusión + logístico + Rössler_x
↓
[4. PREPARACIÓN DE DATOS]
- Empaquetar vector_cifrado
- Incluir trayectorias completas (x, y, z, t)

```

- Incluir parámetros de imagen
- Incluir parámetros de sistemas caóticos

↓

[5. TRANSMISIÓN MQTT CON TLS]

- Conectar a broker (puerto 8883)
- Autenticar usuario/contraseña
- Verificar certificado CA
- Publicar parámetros (topic: keys)
- Publicar datos cifrados (topic: data)

↓

[6. GENERACIÓN DE MÉTRICAS]

- Calcular tiempos de ejecución
- Generar gráficas comparativas
- Calcular distancia de Hamming
- Guardar resultados

↓

[FIN]